

PRACTICAL - 5

Implementation of 0/1 Knapsack using Dynamic Programming

Aim of the Practical:

To solve the 0/1 Knapsack problem using dynamic programming and analyze the DP table construction.

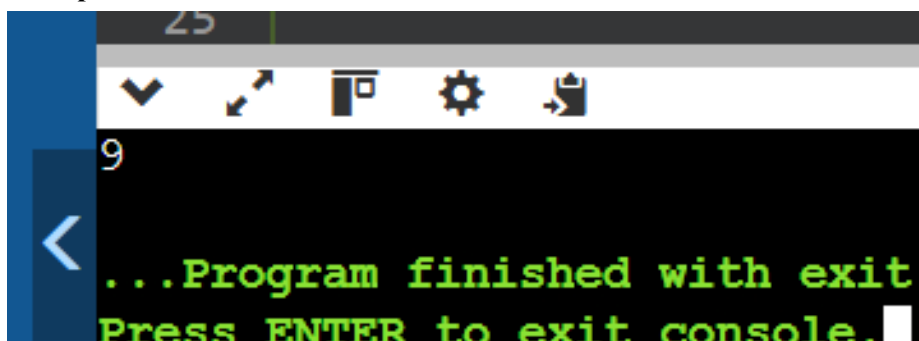
Algorithm:

1. Let $dp[i][w]$ denote the best value using the first i items and capacity w .
2. Initialize row 0 and column 0 with 0.
3. If $wt[i-1] > w$, set $dp[i][w] = dp[i-1][w]$.
4. Otherwise set $dp[i][w] = \max(dp[i-1][w], value[i-1] + dp[i-1][w-wt[i-1]])$.
5. Return $dp[n][W]$.

Program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int knapsack(int W, const vector<int>& wt, const vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (wt[i - 1] <= w) {
                dp[i][w] = max(dp[i][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            }
        }
    }
    return dp[n][W];
}
int main() {
    vector<int> wt = {1, 3, 4, 5};
    vector<int> val = {1, 4, 5, 7};
    int W = 7;
    cout << knapsack(W, wt, val);
}
```

Output:



Time Analysis:

Derivation of Time Complexity:

State Generation:

- Let n represent the number of items and W represent the knapsack capacity.
- The dynamic programming table requires $(n + 1)$ rows and $(W + 1)$ columns, generating a total of $(n + 1)(W + 1)$ states.

Time Complexity:

The algorithm visits every non-base state exactly once

Because each state requires only constant-time comparisons and addition operations, the total time derivation is $c \cdot n \cdot (W + 1) + c_0$.

This simplifies to a final time complexity of $\Theta(nW)$.

Final Time Analysis:

- Time Complexity: $\Theta(nW)$.
- Space Complexity: $\Theta(nW)$.

Result:

The 0/1 Knapsack problem was solved using dynamic programming, and its complexity was derived from the number of DP states.